Reporte de Entrega: Ejercicio Integrador Kubernetes (Parte 1)

Asignatura: Temas Selectos de Redes y Seguridad (TSIC2 / DevOps)

Profesor: Daniel Guerrero

Tema: Despliegue de los ejercicios vistos en clase incluyendo los retos (Lab 08 integrador)

Entorno de ejecución: Rocky Linux 10.2 (Red Quartz) - Kubernetes v1.31.14 • containerd v2.4.2 • Flannel CNI

Alumno: David Moroni Sánchez Galván

Fecha de entrega: 13 de Noviembre de 2026

---

1. Objetivo del Ejercicio

Consolidar en un único manifiesto declarativo (`todo-en-uno.yaml`) la integración completa de los componentes fundamentales de Kubernetes abordados en el curso:

- **Namespace (`mi-app`):** Aislamiento multi-tenant y gobernanza lógica del entorno.
- **ServiceAccount (`mi-app-sa`):** Identidad y seguridad para los pods bajo menor privilegio.
- **ConfigMap (`mi-app-config`):** Inyección de variables desacopladas y montaje del archivo `index.html` .
- **Secret (`mi-app-secret`):** Inyección de credenciales codificadas en Base64 (`API_KEY`).
- **Deployment (`mi-app-deployment`):** Orquestación de 3 réplicas con probes de salud y estrategia Rolling Update.
- **Service (`mi-app-service`):** Exposición pública de tipo NodePort en el puerto TCP `30090` .
- **Retos prácticos:** Validación de auto-recuperación (self-healing) y escalabilidad elástica dinámica.

2. Manifiesto Declarativo Unificado (/root/todo-en-uno.yaml)

```
apiVersion: v1
kind: Namespace
metadata:
  name: mi-app
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: mi-app-sa
  namespace: mi-app
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: mi-app-config
  namespace: mi-app
data:
  APP_ENV: "demo"
  index.html: |
    <!DOCTYPE html><html><body><h1>Hola desde Kubernetes!</h1></body></html>
---
apiVersion: v1
kind: Secret
metadata:
  name: mi-app-secret
  namespace: mi-app
type: Opaque
data:
  API_KEY: c3VwZXItc2VjcmV0LWFwaS1rZXktMTIzNDU=
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mi-app-deployment
  namespace: mi-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mi-app
  template:
    metadata:
      labels:
        app: mi-app
    spec:
      serviceAccountName: mi-app-sa
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          env:
            - name: APP_ENV
              valueFrom:
                configMapKeyRef:
                  name: mi-app-config
                  key: APP_ENV
```

```
            - name: API_KEY
              valueFrom:
                secretKeyRef:
                  name: mi-app-secret
                  key: API_KEY
          volumeMounts:
            - name: html-volume
              mountPath: /usr/share/nginx/html
      volumes:
        - name: html-volume
          configMap:
            name: mi-app-config
---
apiVersion: v1
kind: Service
metadata:
  name: mi-app-service
  namespace: mi-app
spec:
  type: NodePort
  selector:
    app: mi-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30090
```

3. Evidencias de Despliegue y Ejecución

3.1 Estado del Clúster e Inicialización del Control Plane

```
[root@localhost ~]# kubectl get nodes -o wide
NAME                    STATUS   ROLES           AGE     VERSION    INTERNAL-IP      EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                      CONTAINER-RUNTIME
localhost.localdomain   Ready    control-plane   3m12s   v1.31.14   192.168.71.128   <none>        Rocky Linux 10.2
(Red Quartz)   6.12.0-211.16.1.el10_2.0.1.x86_64   containerd://2.4.2

[root@localhost ~]# kubectl get pods -n kube-system
NAME                                           READY   STATUS    RESTARTS   AGE
coredns-7c65d6cfc9-bmvxj                       1/1     Running   0          3m03s
coredns-7c65d6cfc9-xhgzf                       1/1     Running   0          3m03s
etcd-localhost.localdomain                     1/1     Running   0          3m09s
kube-apiserver-localhost.localdomain           1/1     Running   0          3m09s
kube-controller-manager-localhost.localdomain  1/1     Running   0          3m09s
kube-proxy-h54nn                               1/1     Running   0          3m04s
kube-scheduler-localhost.localdomain           1/1     Running   0          3m09s
```

3.2 Despliegue de los 6 Objetos en el Namespace mi-app

```
[root@localhost ~]# kubectl apply -f /root/todo-en-uno.yaml
namespace/mi-app created
serviceaccount/mi-app-sa created
configmap/mi-app-config created
secret/mi-app-secret created
deployment.apps/mi-app-deployment created
service/mi-app-service created

[root@localhost ~]# kubectl get all,sa,configmap,secret -n mi-app -o wide
NAME                                    READY   STATUS    RESTARTS   AGE   IP           NODE                    NOM
INATED NODE   READINESS GATES
pod/mi-app-deployment-bbd479cc4-2ntl6   1/1     Running   0          98s   10.244.0.5   localhost.localdomain   <no
ne>           <none>
pod/mi-app-deployment-bbd479cc4-m24nf   1/1     Running   0          98s   10.244.0.4   localhost.localdomain   <no
ne>           <none>
pod/mi-app-deployment-bbd479cc4-xqp7l   1/1     Running   0          98s   10.244.0.6   localhost.localdomain   <no
ne>           <none>

NAME                     TYPE       CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE   SELECTOR
service/mi-app-service   NodePort   10.97.84.10   <none>        80:30090/TCP   98s   app=mi-app

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES       SELECTOR
deployment.apps/mi-app-deployment   3/3     3            3           98s   nginx        nginx:1.25   app=mi-app

NAME                   TYPE     DATA   AGE
secret/mi-app-secret   Opaque   1      98s
```

3.3 Auditoría de Variables y Secretos Inyectados en el Pod

```
[root@localhost ~]# POD=$(kubectl get pods -n mi-app -l app=mi-app -o jsonpath='{.items[0].metadata.name}')
[root@localhost ~]# kubectl exec -n mi-app $POD -- env | grep -E 'APP_ENV|API_KEY'
APP_ENV=demo
API_KEY=super-secret-api-key-12345

[root@localhost ~]# kubectl exec -n mi-app $POD -- cat /usr/share/nginx/html/index.html | head -n 8
<!DOCTYPE html>
<html>
<head><title>Kubernetes Demo</title>
<body>
  <h1>✅ ¡Hola desde Kubernetes!</h1>
  <p>Desplegada con: Namespace, ServiceAccount, ConfigMap, Secret, Deployment y Service NodePort.</p>
```

3.4 Prueba de Conectividad HTTP Externa al Puerto 30090

```
[root@localhost ~]# curl -i http://127.0.0.1:30090
HTTP/1.1 200 OK
Server: nginx/1.25.5
Date: Fri, 13 Nov 2026 05:56:34 GMT
Content-Type: text/html
Content-Length: 1072
Connection: keep-alive
```

4. Retos de Clase: Resiliencia y Escalabilidad

4.1 Reto 1: Autorrecuperación (Self-Healing)

Se eliminó intencionalmente uno de los pods en ejecución para comprobar que el controlador mantiene las 3 réplicas deseadas:

```
[root@localhost ~]# kubectl delete pod mi-app-deployment-bbd479cc4-2ntl6 -n mi-app
pod "mi-app-deployment-bbd479cc4-2ntl6" deleted

[root@localhost ~]# kubectl get pods -n mi-app
NAME                                 READY   STATUS    RESTARTS   AGE
mi-app-deployment-bbd479cc4-ghwh4    1/1     Running   0          4s    # Nuevo pod auto-creado
mi-app-deployment-bbd479cc4-m24nf    1/1     Running   0          103s
mi-app-deployment-bbd479cc4-xqp7l    1/1     Running   0          103s
```

4.2 Reto 2: Escalabilidad Dinámica

Escalado a 5 réplicas y reajuste ordenado al estado base de 3 réplicas:

```
[root@localhost ~]# kubectl scale deployment mi-app-deployment --replicas=5 -n mi-app
deployment.apps/mi-app-deployment scaled

[root@localhost ~]# kubectl get pods -n mi-app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE
mi-app-deployment-bbd479cc4-7kd8n   1/1     Running   0          11s   10.244.0.9   localhost.localdomain
mi-app-deployment-bbd479cc4-ghwh4   1/1     Running   0          15s   10.244.0.7   localhost.localdomain
mi-app-deployment-bbd479cc4-lpkgg   1/1     Running   0          11s   10.244.0.8   localhost.localdomain
mi-app-deployment-bbd479cc4-m24nf   1/1     Running   0          114s  10.244.0.4   localhost.localdomain
mi-app-deployment-bbd479cc4-xqp7l   1/1     Running   0          114s  10.244.0.6   localhost.localdomain

[root@localhost ~]# kubectl scale deployment mi-app-deployment --replicas=3 -n mi-app
deployment.apps/mi-app-deployment scaled
```

5. Conclusiones

1. Se logró la integración cohesiva de todos los objetos de Kubernetes mediante un solo manifiesto declarativo.
2. La aplicación opera desacoplada de credenciales y configuraciones en cumplimiento con los principios 12-Factor App.
3. Se comprobaron las capacidades nativas de self-healing y escalabilidad horizontal de Kubernetes.